UNIVERSIDADE POSITIVO

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE APLICATIVOS MÓVEIS

MERCADO PRESO

Documentação técnica do aplicativo Android — Trabalho 3: persistência com Room

Luciano Diniz Javorski

Matheus Alves de Lima

Marianne Cristina Bueno

CURITIBA — 2026

DOCUMENTAÇÃO TÉCNICA

Mercado Preso

Aplicativo Android com Kotlin, Jetpack Compose e persistência local com Room

Informação	Detalhe
Aplicativo	Mercado Preso
Disciplina	Desenvolvimento de Aplicativos Móveis
Plataforma	Android (Kotlin + Jetpack Compose + Material 3)
Pacote	com.example.myapplication
Repositório	https://github.com/Lucdijord/App-MP
Data da documentação	08 de outubro de 2026

Sumário

1. Introdução.....2

2. Tecnologias Utilizadas.....2

3. Arquitetura e Estrutura do Projeto.....3

4. Sistema de Rotas.....4

5. Telas do Aplicativo.....5

6. Modelo de Dados.....8

7. Interface e Design.....9

8. Como Executar o Projeto.....13

9. Melhorias Futuras.....13

10. Trabalho 3 — Persistência com Room, testes e evidências.....14

11. Considerações Finais.....18

1. Introdução

O Mercado Preso é um aplicativo acadêmico de comércio eletrônico para dispositivos Android, criado como trabalho da disciplina de Desenvolvimento de Aplicativos Móveis. Ele simula a experiência de um pequeno marketplace: o usuário navega por uma tela inicial com busca, banner promocional, categorias e produtos em destaque, consulta o catálogo, vê os detalhes de cada item, escolhe a quantidade, adiciona ao carrinho e gerencia seus endereços de entrega.

O código-fonte e toda a evolução do projeto estão disponíveis no repositório público do GitHub, indicado na capa deste documento. Este material descreve a finalidade do aplicativo, a arquitetura adotada, as telas implementadas e o modo de execução, servindo de guia tanto para avaliação acadêmica quanto para futuras manutenções.

1.1 Objetivos

- **Objetivo geral:** Aplicar, em um projeto completo, os conceitos de desenvolvimento móvel nativo para Android.
- **Interface:** Construir interfaces declarativas com Jetpack Compose e Material 3.
- **Navegação:** Implementar navegação entre várias telas, com passagem de argumentos (por exemplo, o identificador do produto).
- **Estado:** Gerenciar estado local de tela (quantidade, texto de busca) e dados compartilhados (carrinho).
- **Organização:** Organizar o código em arquivos coesos e com responsabilidades bem definidas.

1.2 Escopo

O projeto mantém a navegação, o catálogo, o carrinho e o cadastro de endereços. No Trabalho 3, produtos e endereços passaram a ser armazenados localmente no SQLite por meio do Room. O carrinho continua em memória, sem persistência entre execuções. Não há integração com servidor, pagamentos ou autenticação.

2. Tecnologias Utilizadas

O aplicativo foi desenvolvido em Kotlin com Jetpack Compose e Material 3. Nesta etapa, foram adicionados Room, SQLite, KSP e Coroutines para realizar operações de banco de dados sem bloquear a interface.

Tecnologia	Uso no projeto
Kotlin	Linguagem principal de todo o aplicativo.
Jetpack Compose	Construção declarativa da interface (funções @Composable).
Material 3	Componentes visuais: Scaffold, TopAppBar, Button, OutlinedTextField, Icon, NavigationBar.
Navigation Compose	Navegação entre telas por rotas, com argumentos dinâmicos.
Android Studio	Ambiente de desenvolvimento, emulador e depuração.

Tecnologia	Uso no projeto
Git e GitHub	Controle de versão e hospedagem do código-fonte (branch main).

3. Arquitetura e Estrutura do Projeto

O projeto mantém uma organização simples: as telas ficam no pacote com.example.myapplication.telas; as entidades, DAOs e AppDatabase ficam em com.example.myapplication.banco. MainActivity, AppNavigation, Rotas e DadosCarrinho permanecem no pacote principal.

3.1 Arquivos principais

Arquivo	Responsabilidade
MainActivity.kt	Ponto de entrada do aplicativo; define o conteúdo Compose e inicia a navegação.
AppNavigation.kt	Grafo de navegação: associa cada rota à sua tela e controla a barra inferior.
Rotas.kt	Objeto com as constantes de rota e funções auxiliares para montar rotas com parâmetros.
HomeScreen.kt	Tela inicial (Início): logotipo, busca, banner, categorias e destaques.
CatalogScreen.kt	Listagem do catálogo de produtos.
DetalheProdutoScreen.kt	Detalhes do produto, seleção de quantidade e valor total.
AdicionarProdutoScreen.kt	Formulário de cadastro de novo produto.
CarrinhoScreen.kt	Exibição dos itens adicionados ao carrinho.
DadosCarrinho.kt	Armazenamento e regras dos itens do carrinho.
Produto.kt	Entidade Room com os dados persistidos no SQLite.
ProfileScreen.kt	Tela de perfil do usuário e acesso aos endereços.
EnderecosScreen.kt	Lista de endereços cadastrados, com opção de exclusão.
AdicionarEnderecoScreen.kt	Formulário de cadastro de novo endereço.
DetalheEnderecoScreen.kt	Detalhes de um endereço específico.
Endereco.kt	Entidade Room com os dados persistidos no SQLite.

3.2 Fluxo de navegação

A navegação principal acontece por uma barra inferior com quatro destinos: Início, Catálogo, Carrinho e Perfil. A partir deles, o usuário alcança as telas secundárias — detalhe do produto, adicionar produto, lista de endereços, adicionar endereço e detalhe do endereço — e retorna por meio do botão Voltar.

> **Visão resumida do fluxo**

> Início → Catálogo → Detalhe do Produto → Carrinho | Perfil → Meus Endereços → Adicionar / Detalhar Endereço

4. Sistema de Rotas

Todas as rotas do aplicativo são declaradas em um único objeto Kotlin chamado Rotas. Essa centralização evita textos repetidos e propensos a erro ao longo do código: cada tela e cada ação de navegação referenciam a mesma constante. A própria IDE indica o uso intenso do objeto — 26 referências em todo o projeto.

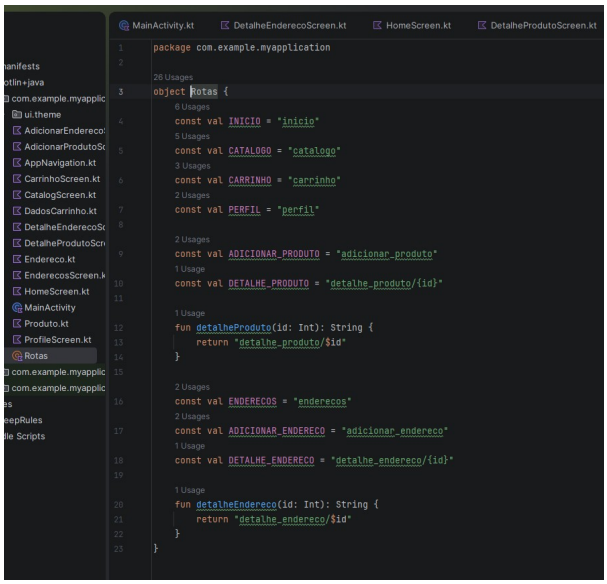


Figura 1 – Objeto Rotas (Rotas.kt) com as constantes e funções de rota.

4.1 Rotas disponíveis

Constante	Valor	Finalidade
INICIO	inicio	Tela inicial do aplicativo.
CATALOGO	catalogo	Lista de produtos.
CARRINHO	carrinho	Carrinho de compras.
PERFIL	perfil	Perfil do usuário.
ADICIONAR_PRODUTO	adicionar_produto	Cadastro de produto.
DETALHE_PRODUTO	detalhe_produto/{id}	Detalhes de um produto (recebe o id).
ENDERECOS	enderecos	Lista de endereços.
ADICIONAR_ENDERECO	adicionar_endereco	Cadastro de endereço.
DETALHE_ENDERECO	detalhe_endereco/{id}	Detalhes de um endereço (recebe o id).

4.2 Rotas com parâmetros

Para as telas de detalhe, a rota possui um marcador {id}, que o Navigation Compose substitui pelo identificador do item. Duas funções auxiliares montam o caminho final de forma segura, usando interpolação de texto do Kotlin:

```
fun detalheProduto(id: Int): String {
    return "detalhe_produto/$id"
}

fun detalheEndereco(id: Int): String {
    return "detalhe_endereco/$id"
}
```

5. Telas do Aplicativo

5.1 Tela Inicial (Início)

A tela inicial é a porta de entrada do aplicativo e reúne os principais atalhos de compra em uma única rolagem vertical. De cima para baixo, ela apresenta:

- **Identidade visual:** logotipo “MP” em destaque, centralizado em um quadrado de 80 dp com fundo rosado;
- **Pesquisa:** campo de busca com o rótulo “Buscar produtos…”, de linha única e largura total;
- **Promoção:** banner “Ofertas imperdíveis” com o texto “Descontos especiais para você!” e o botão “Ver ofertas”, que leva ao catálogo;
- **Categorias:** atalhos para Eletro, Casa, Beleza e Mais;
- Destaques para você: exibe produtos cadastrados e consultados no banco local, com acesso aos detalhes e ao carrinho.

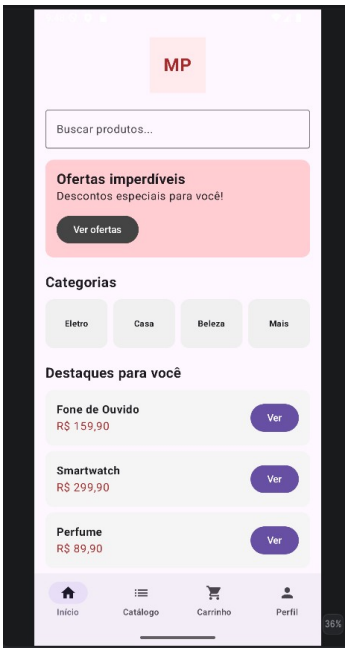


Figura 2 – Tela Início executando no emulador Android.

A tela é implementada pela função HomeScreen, que recebe o evento onCatalogoClick por parâmetro. Esse padrão, conhecido como elevação de estado (state hoisting), mantém a tela desacoplada da navegação: quem decide para onde ir é o AppNavigation. O texto digitado na busca é guardado em uma variável de estado (pesquisa), atualizada a cada alteração do campo.

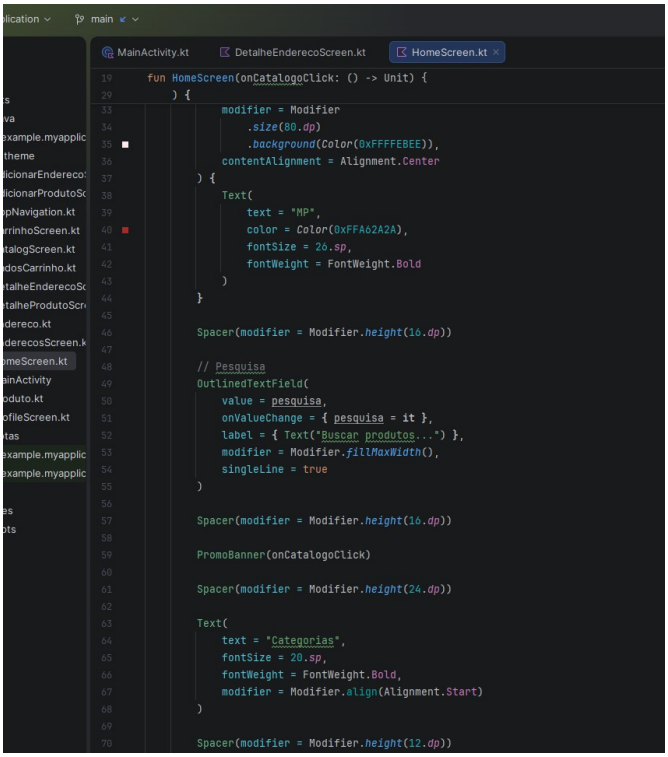


Figura 3 – Trecho do código de HomeScreen.kt (logotipo, busca, banner e categorias).

5.2 Detalhes do Produto

Ao tocar em “Ver”, o usuário abre a tela de detalhes. A tela recebe o produtoId pela rota e consulta o produto no banco local usando o ProdutoDao. Assim, os detalhes apresentados correspondem aos dados salvos pelo Room.

Quando o produto é encontrado, a tela apresenta nome, categoria e preço, além das opções de quantidade e de adicionar ao carrinho.

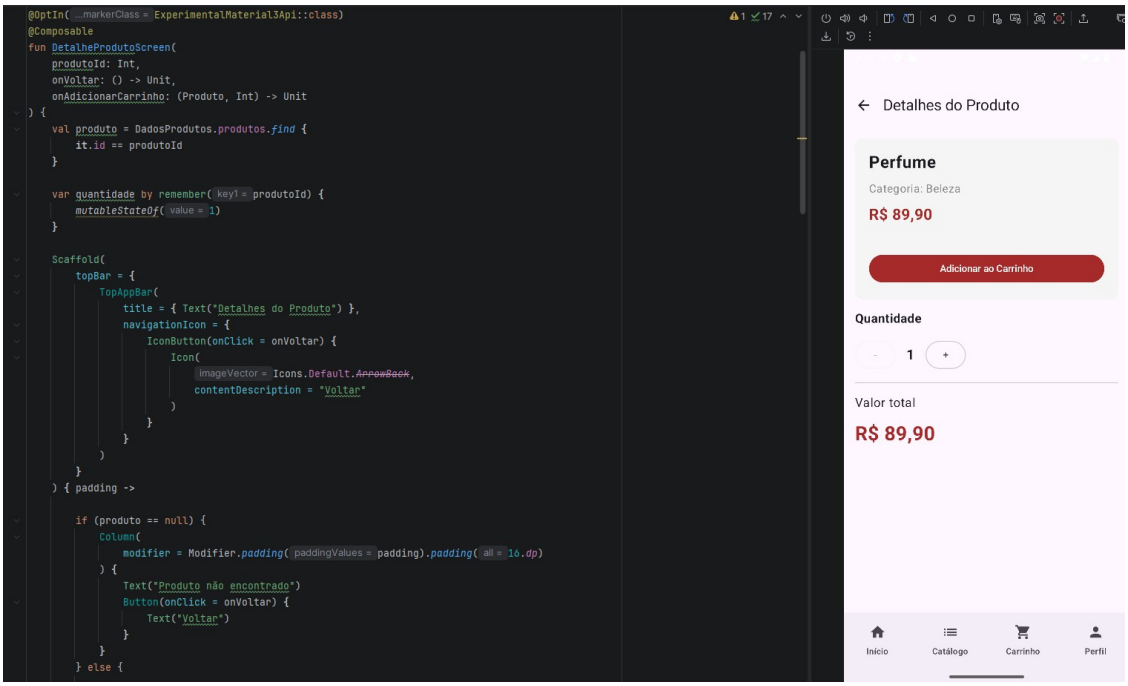


Figura 4 – Código de DetalheProdutoScreen.kt (esquerda) e tela em execução (direita).

Pontos de implementação relevantes:

- **Material 3:** a anotação @OptIn(ExperimentalMaterial3Api::class) habilita o uso do TopAppBar do Material 3;
- **Estado local:** a variável quantidade usa remember(produtoId) { mutableStateOf(1) }, reiniciando em 1 sempre que o produto muda;
- **Estrutura:** o Scaffold organiza a barra superior com o título “Detalhes do Produto” e o ícone de voltar com descrição de acessibilidade;
- **Comunicação:** o callback onAdicionarCarrinho(Produto, Int) entrega ao chamador o produto e a quantidade escolhida.

5.3 Meus Endereços

Acessível pelo Perfil, a tela “Meus Endereços” lista os endereços cadastrados em cartões, mostrando rua, número, bairro e cidade. Cada cartão possui um ícone vermelho de lixeira para exclusão, e o botão “Adicionar Endereço” abre o formulário de cadastro. Os exemplos exibidos são Rua das Flores, nº 123, Centro, Curitiba e Avenida Brasil, nº 500, Batel, Curitiba.

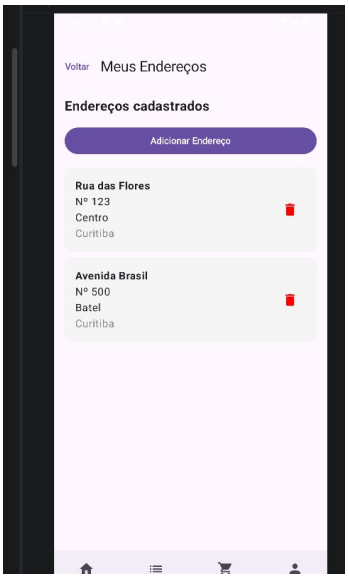


Figura 5 – Tela Meus Endereços com dois endereços cadastrados.

5.4 Demais telas

Tela	Descrição
Catálogo	Apresenta todos os produtos disponíveis e permite abrir o detalhe de cada um.
Carrinho	Reúne os itens adicionados com suas quantidades e valores.
Perfil	Concentra as informações do usuário e o acesso à gestão de endereços.
Adicionar Produto	Formulário para cadastrar novos produtos na lista.
Adicionar Endereço	Formulário para cadastrar um novo endereço de entrega.
Detalhe do Endereço	Exibe as informações completas de um endereço selecionado.

6. Modelo de Dados

Os dados principais são Produto e Endereco, agora definidos como entidades do Room (@Entity), com chave primária de geração automática. Os registros são salvos no SQLite. Os itens do carrinho continuam em uma lista temporária em memória.

Entidade	Campos	Observações
Produto	id, nome, categoria, preço	Armazenado no SQLite por meio do Room.
Endereço	id, rua, número, bairro, cidade	Armazenado no SQLite por meio do Room.
Item do carrinho	produto, quantidade	Gerenciado em DadosCarrinho; alimenta a tela do carrinho.

7. Interface e Design

O visual segue as diretrizes do Material 3, com cantos arredondados, cartões claros e hierarquia tipográfica definida. Após a revisão visual, a paleta passou a priorizar o vermelho-vinho, tons rosados e branco, mantendo a identidade do aplicativo em botões, ícones e barras de navegação. Os principais critérios adotados foram:

- **Hierarquia:** títulos em negrito e valores monetários em destaque, facilitando a leitura rápida;
- **Consistência:** a mesma estrutura de cartão e botão arredondado em todas as telas;
- **Navegação previsível:** barra inferior sempre disponível, com ícones e rótulos para Início, Catálogo, Carrinho e Perfil;
- **Acessibilidade:** ícones com contentDescription e botões com área de toque adequada;
- **Ritmo visual:** espaçamentos múltiplos de 4 dp (12, 16, 24 dp) entre os elementos.

Conforme foi debatido e analisado em sala de aula, e seguindo a recomendação do professor, decidimos atualizar as telas do aplicativo para apresentar um visual mais moderno, organizado e chamativo. Optamos pelo vermelho como cor principal por ser a cor representada na identidade visual e no logotipo desenvolvidos no Figma. A nova paleta combina vermelho-vinho, branco e tons claros de rosa, trazendo mais consistência aos botões, à barra superior, à barra inferior e aos cartões de produtos.
As telas modificadas ficaram da seguinte forma:

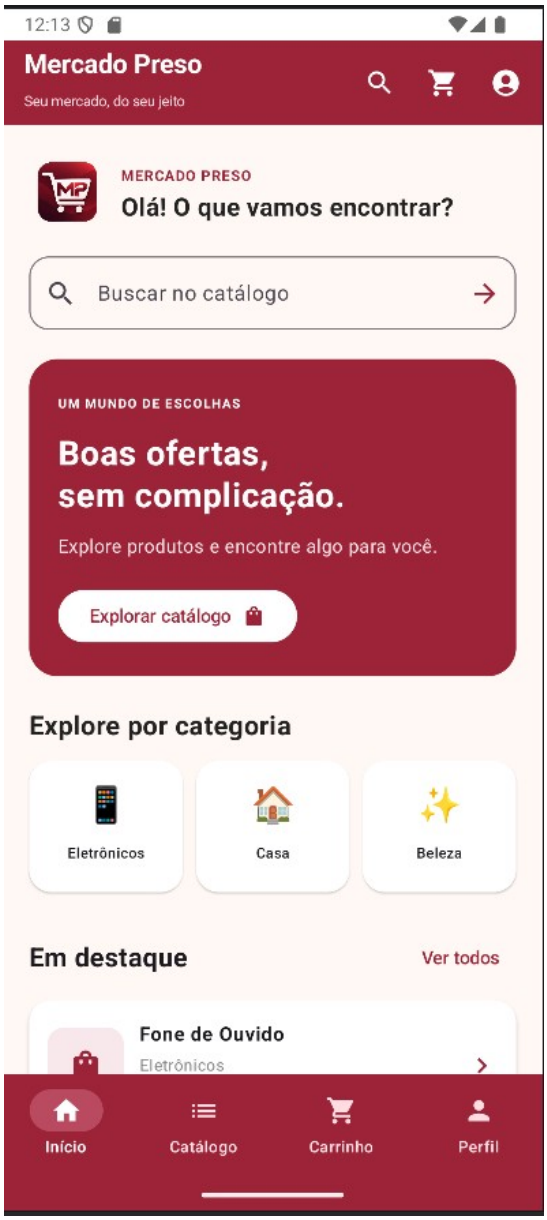


Figura 6 – Tela inicial após a atualização visual.



Figura 7 – Catálogo de produtos após a atualização visual.

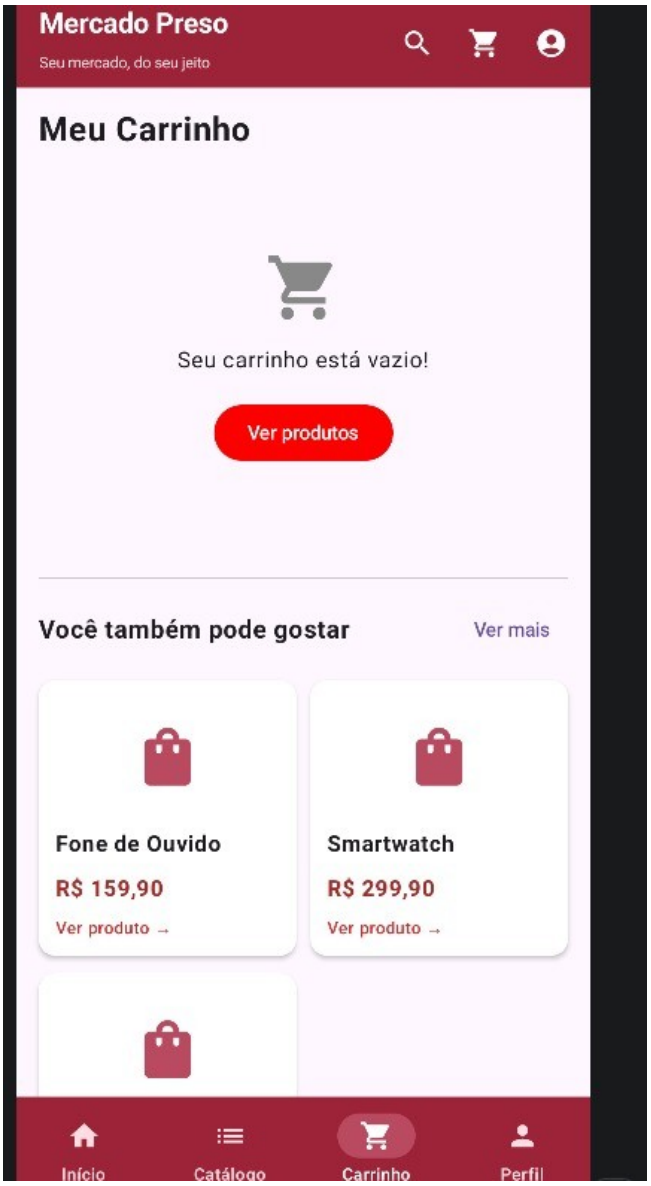


Figura 8 – Carrinho de compras com sugestões de produtos.

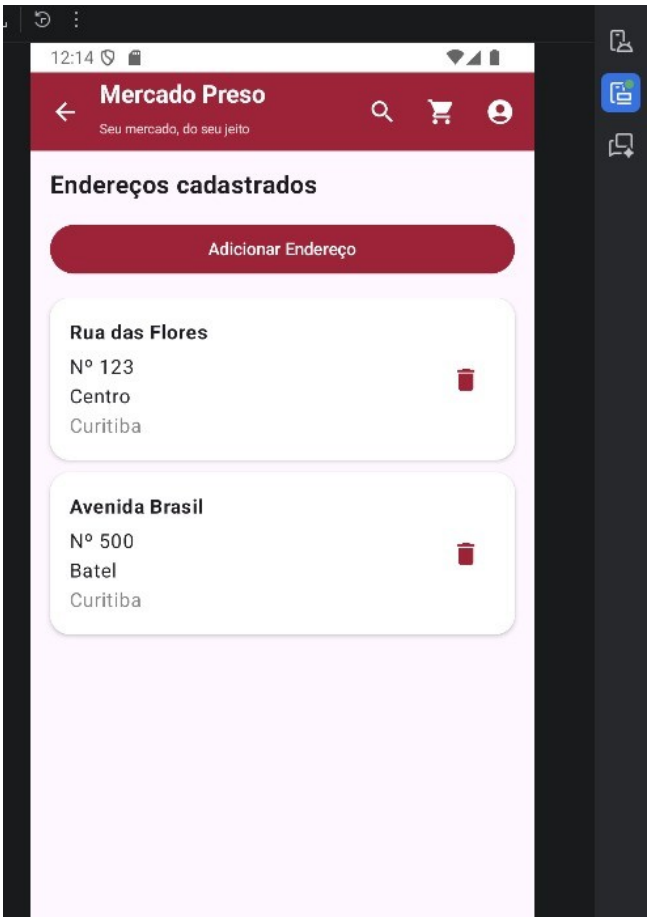


Figura 9 – Tela de endereços cadastrados com a nova paleta.

8. Como Executar o Projeto

8.1 Requisitos

- Android Studio em versão recente, com o SDK Android instalado.
- Emulador Android configurado ou dispositivo físico com depuração USB ativada.
- Git para clonar o repositório.

8.2 Passo a passo

1. Clone o repositório: git clone https://github.com/Lucdijord/MercadoPreso-Trabalho3
2. Abra a pasta do projeto no Android Studio e aguarde a sincronização do Gradle.
3. Selecione um emulador ou dispositivo na barra superior.
4. Clique em Run (botão verde de execução) para compilar e instalar o aplicativo.
5. Navegue pela barra inferior para explorar as telas.

9. Melhorias Futuras

Como o projeto é acadêmico e continua evoluindo, estas são as extensões naturais para as próximas versões:

- Persistência do carrinho: possibilidade de salvar os itens do carrinho em uma próxima etapa. Produtos e endereços já utilizam Room nesta versão.
- **Integração com servidor:** Consumo de uma API para produtos, estoque e pedidos.
- **Autenticação:** Cadastro, login e perfil real do usuário.
- Busca e filtros: aprimorar os recursos de pesquisa e filtragem do catálogo.
- **Checkout:** Fluxo de finalização de compra, com escolha de endereço e forma de pagamento.
- **Qualidade:** Testes unitários e de interface com Compose Testing.
- **Visual:** Inclusão de imagens reais dos produtos e suporte a tema escuro.

10. Trabalho 3 — Persistência de dados com Room

Nesta etapa, mantivemos as telas e funcionalidades do Trabalho 2 e substituímos o armazenamento temporário de produtos e endereços por um banco de dados local. Com isso, as informações continuam disponíveis mesmo depois de fechar e abrir o aplicativo.

10.1 Estrutura dos arquivos

banco/Produto.kt e banco/Endereco.kt — Definem as entidades e os campos das tabelas.
banco/ProdutoDao.kt e banco/EnderecoDAO.kt — Contêm os métodos de inserir, consultar, atualizar e excluir.
banco/AppDatabase.kt — Configura o Room e fornece acesso aos DAOs por uma instância única.
telas/*.kt — Exibem as informações e chamam os DAOs conforme a ação do usuário.
DadosCarrinho.kt — Mantém os itens do carrinho temporariamente em memória.

10.2 Entidades e DAOs

As classes Produto e Endereco usam @Entity e @PrimaryKey(autoGenerate = true). O identificador é criado automaticamente pelo banco. Os DAOs utilizam @Insert, @Query, @Update e @Delete para implementar o CRUD. Também há consultas por ID, usadas nas telas de detalhes.

10.3 AppDatabase e Coroutines

A classe AppDatabase informa quais entidades fazem parte do banco e permite acessar os DAOs. Ela utiliza o padrão singleton, evitando criar várias instâncias desnecessárias do banco. As operações são executadas com funções suspend e Coroutines no Dispatchers.IO, para não bloquear a interface do aplicativo. Nas telas, LaunchedEffect ajuda a carregar os dados quando necessário.

10.4 Exemplo do fluxo de cadastro

O usuário preenche nome, preço e categoria na tela de cadastro.
A tela valida os campos e cria um objeto Produto.
O ProdutoDao insere o registro no banco por meio do Room.
Ao consultar o catálogo ou a Home, os produtos são carregados do SQLite e apresentados na tela.

10.5 Testes realizados

Foram realizados testes de cadastro, consulta e exclusão de produtos e endereços, além da atualização do complemento de um endereço. Depois, o aplicativo foi encerrado usando a opção “Forçar parada” do Android e aberto novamente. Os produtos, endereços e alterações continuaram salvos. O carrinho foi esvaziado após encerrar o processo, pois não foi incluído na persistência desta etapa.

10.6 Dificuldades e soluções

Durante a adaptação, foi necessário trocar as consultas às listas em memória pelas consultas ao Room. Também foram corrigidos problemas de compilação relacionados a referências de recursos do Android e ajustados os imports após a organização dos arquivos nos pacotes banco e telas. Ao final, o projeto compilou e os testes de persistência foram realizados.

10.7 Evidências dos testes do Trabalho 3

As figuras a seguir registram a organização do código e um teste de persistência de produto. Após cadastrar a TV por R$ 100,00, o aplicativo foi interrompido nas configurações do Android e, ao ser aberto novamente, o produto continuou no catálogo.

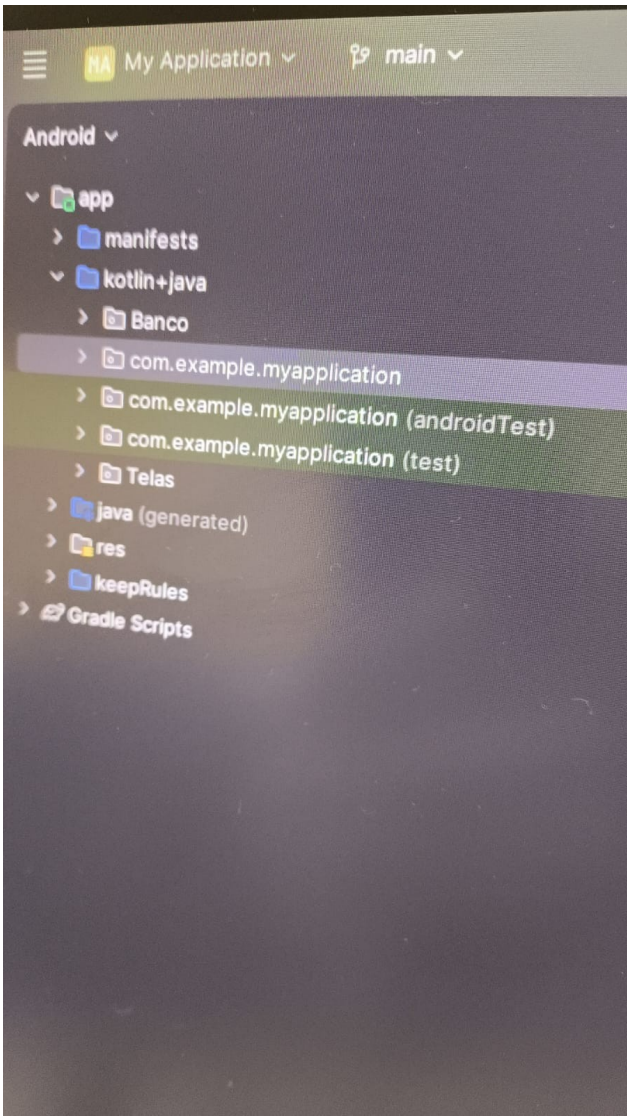


Figura 10 – Organização dos pacotes Banco e Telas no Android Studio.

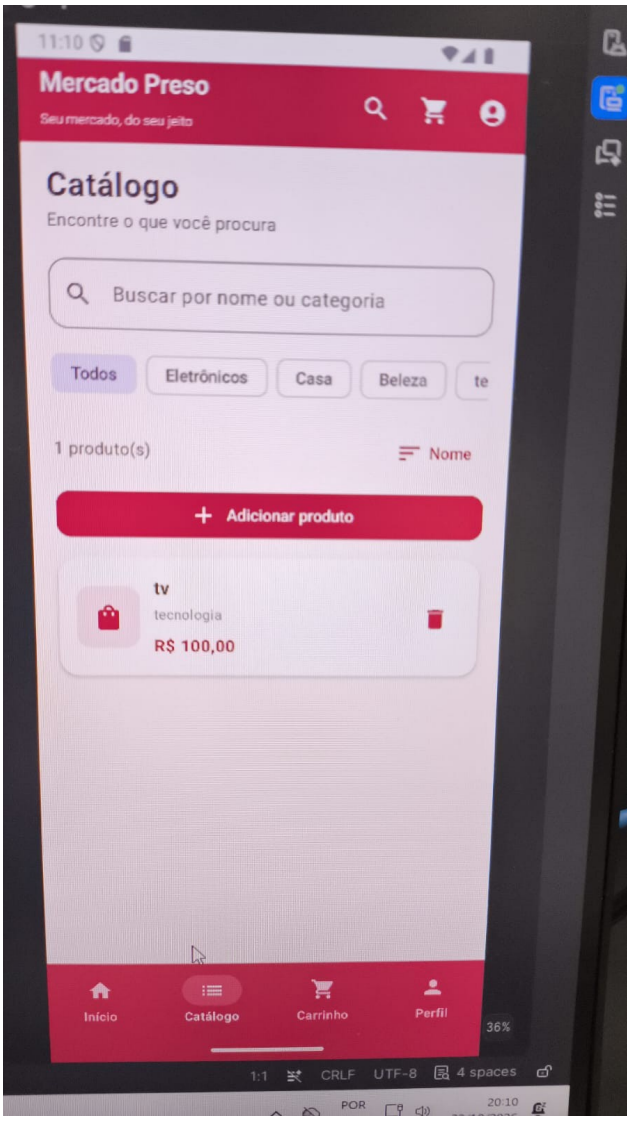


Figura 11 – Produto TV cadastrado e exibido no catálogo.

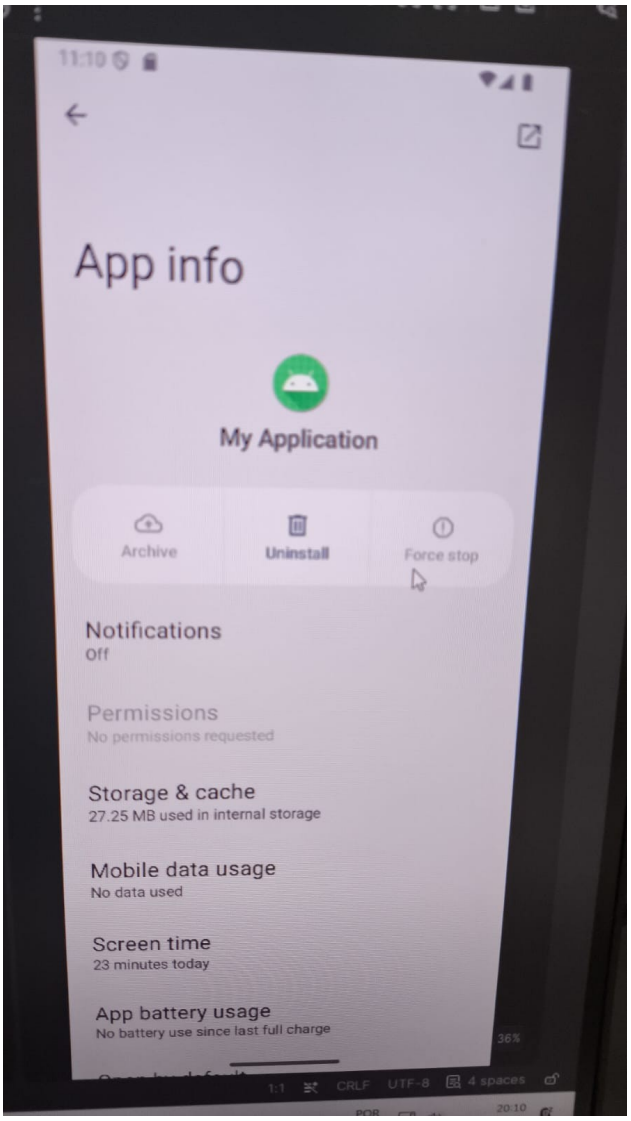


Figura 12 – Tela de informações do aplicativo no Android, utilizada para interromper sua execução.

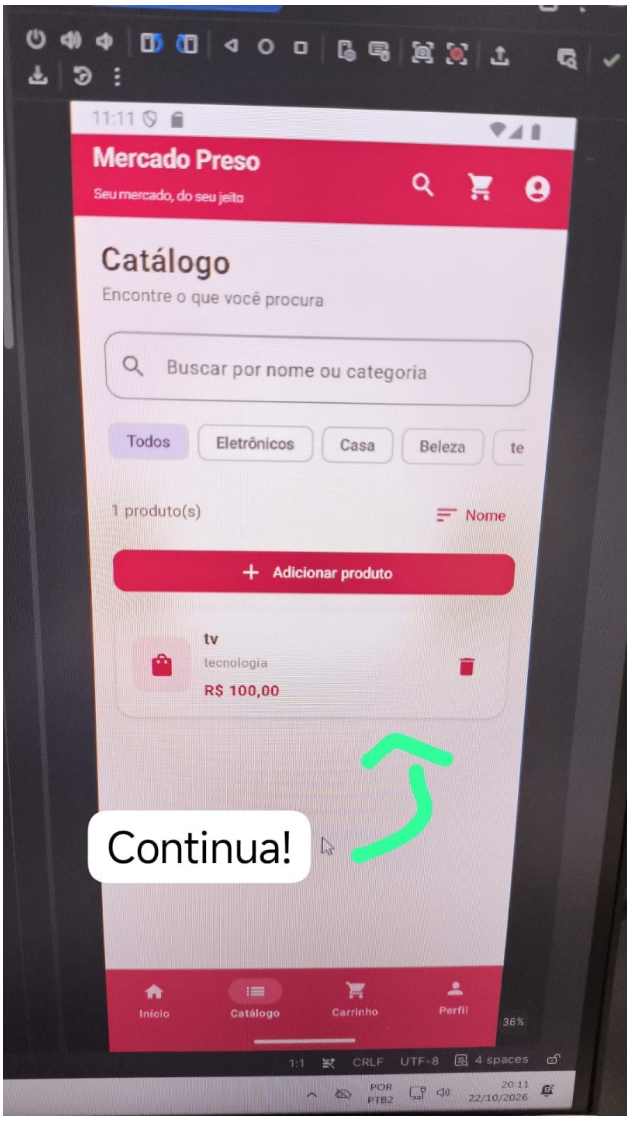


Figura 13 – Produto TV ainda presente no catálogo após reabrir o aplicativo, demonstrando a persistência.

11. Considerações Finais

O Mercado Preso reúne interfaces com Jetpack Compose, navegação por rotas e gerenciamento de estado. No Trabalho 3, o projeto passou a contar também com persistência local de produtos e endereços usando Room, com operações de cadastro, consulta, edição e exclusão testadas após reiniciar o aplicativo.

Repositório do projeto

https://github.com/Lucdijord/App-MP (verificar e substituir pelo repositório do Trabalho 3)